



How re-forge is *different*.

POSITIONING

The model-agnostic, tool-agnostic *memory, skill and audit layer* that makes any AI coding agent remember, improve, and prove its work.

We are **not another assistant** competing with Copilot or Cursor. We are the layer that makes whichever assistant a team already uses accumulate institutional knowledge, forge personalized skills, and pass adversarial quality gates before anything ships at high confidence.

01 The category: we are a layer on top of your stack

Every layer below us is something a team already owns. re-forge does not replace any of it. It sits on top, turns each tool's activity into **durable team memory and forged skills**, and gates output before it ships. We win regardless of which model or which assistant wins.

RE-FORGE	Memory + skill + audit layer. Model-agnostic, tool-agnostic. Compounds per team across every tool below.
ASSISTANTS	Copilot, Cursor, Codex, Claude Code, Amazon Q. The agents that write the code.
FRAMEWORKS	LangGraph, AutoGen, CrewAI. Construction kits for building agents from parts.
OBSERVABILITY	LangSmith, Phoenix, Langfuse, Helicone. Read-only telemetry, after the fact.
MEMORY INFRA	Mem0, Letta, Zep. General-purpose memory primitives a developer integrates.

02 Where everyone else is complementary, and where we win

CATEGORY	WHAT THEY DO	WHY RE-FORGE IS COMPLEMENTARY	WHERE WE WIN
Assistants & editors Copilot · Cursor · Codex · Amazon Q	Write the code: in-editor completion, chat, agentic edits. Locked to their own surface.	They are the agent that writes; we are the shared brain over all of them . Two assistants are two amnesiacs; we span both.	Cross-tool by design. Memory that compounds per team, not per vendor. Adversarial gates an autocomplete-first product never ships.
Agent frameworks LangGraph · AutoGen · CrewAI	Construction kits to build agents from scratch. A blank canvas of orchestration parts.	We are a finished, hardened operating procedure , not a toolkit. You could build a re-forge on a framework, the way you build a CRM on a database.	Time-to-value: works out of the box. Built-in adversarial gates and a 5-dimension evaluator rubric, plus a file-locked concurrent-merge most teams never build right.
Observability & evals LangSmith · Phoenix · Langfuse · Helicone	Trace and log what happened after the fact. Read-only telemetry and eval dashboards.	They tell you what happened; we change what happens before output ships and write the lesson back to memory. Our evidence files can feed their backend.	Active correction, not passive logging. A closed reflect-and-curate loop turns traces into reusable skills. Quality enforced in-line, not inspected later.
Memory infrastructure Mem0 · Letta · Zep	General-purpose memory primitives: key-value or vector stores of facts a developer integrates.	The only genuine overlap, and even here the shape differs. They store; we adjudicate. We can sit on top of a primitive like theirs.	Vertical, not horizontal. Skills as a first-class output, not just stored facts. Quality gating bundled with memory. Safe shared team memory under parallel writes.

Why it is a company, not a *feature*.

03 The moat: three layers that compound together

No single layer is unbreachable. The defensibility is the **compounding interaction of all three**: a competitor must replicate the data, the agnostic architecture, and the quality system at once, while the incumbent's per-team asset keeps compounding every week of use.

LAYER 1

Team-memory data network effect

Every session writes lessons back via generate, reflect, curate. Memory accumulates **per team** and transfers across that team's projects, which is data no base model and no competitor possesses. A per-account switching cost that grows monotonically with use: the Datadog and Snowflake pattern.

LAYER 2

Model and tool agnostic

We win regardless of which model or assistant wins, because we sit **above both**. If a frontier model leapfrogs the rest, we run on it. If a team splits across Cursor and Codex, we span them. The hard part, the concurrent-merge primitive, is already shipped and exercised by green CI.

LAYER 3

Switching cost that compounds

Months of accumulated memory plus a library of forged, personalized skills cannot move to a competitor without abandoning a custom-fitted asset. **Raw facts export; the forged skills, curated lessons and evidence trail do not.** The moat is the compounding fit, not the storage format.

04 Why this is a company, not a feature

THE OBJECTION

"Isn't memory just a feature an incumbent bolts on? And won't Cursor or Copilot simply build this?"

Fair to ask. We answer it plainly, without spin.

THE REBUTTAL

Memory looked like a feature until **Mem0 raised \$24M** and AWS named it the exclusive memory provider for its Agent SDK. The market is now pricing memory as **infrastructure**. And re-forge is broader than memory alone: it bundles memory, skill-forging and an adversarial quality system into a team-scoped layer that compounds per account. **Features do not produce a per-team data asset that grows with use. Platforms do.**

An incumbent cannot build the agnostic version. An assistant that spans Cursor **and** Copilot **and** Codex cannot be built by any one of them: each is incentivized to lock memory to its own walled garden, and no team using two assistants will trust either vendor's silo. *Our neutrality is exactly what an incumbent cannot offer.*

HONEST Pre-revenue, and today's shipped product runs on Claude Code; the cross-tool monitor is roadmap built on a primitive that already exists. What de-risks it: a top-5 hackathon result with 130+ votes, 685-check green CI, 128 open-source skills across 6 teams, and two ICML 2026 SCALE Workshop papers, all orchestrated with re-forge.

Who else is in the *Skills track*.

THE READ

Three of the four Skills-track teams converged on one thesis: *AI says "done" — but is it?* That is validation, not crowding.

The others shipped **point tools**: one records what the agent did, one guards a single syscall. re-forge is the **operating procedure that produces the work** — and the only one that **compounds** across sessions into memory and forged skills.

05 The three teams we are judged beside, and where we win

TEAM	WHAT THEY SHIP	WHERE RE-FORGE WINS
DoneGraph donegraph.space · \$9/mo · verified	"Your AI Shows Its Work." A passive proof-ledger : captures files, commands, tests and an AI recap letter <i>after</i> the run. Tool-agnostic.	We make it right before it ships. They capture the run after the fact; our agents red-team their own work before they claim done, then write the lesson back to memory.
VeraMesh SDK github · verified	A @guard.protect decorator: blocks sensitive paths, rewrites rm -rf to echo, streams an audit log. One runtime guardrail you wire yourself.	We govern the whole job, not one syscall. A guard can stop a bad command before it runs; it cannot tell right work from wrong. We rule at the verdict layer — fresh re-verification, merge-blocking security verdicts, no "done" without evaluator PASS. Complement, not substitute: we'd happily run a guard underneath.
Investment Detective INFERRED likely vertical skill	Could not verify under that name (same-day submission). By the name, a vertical investment-research skill — one agent, one domain.	Not a rival — a customer. A skill is what you build <i>on</i> the layer. re-forge writes, audits and remembers skills like it; it is the substrate beneath such an app.

THE ONE LINE FOR THE ROOM

In a track full of tools that *prove* or *guard* AI work, re-forge is the only one that *produces* verified work — and the only one that *compounds*.